

Project 10: Job Portal and Recruitment Management System Using Angular & TypeScript

Course

Advanced JavaScript Frameworks (Angular)

Submitted By:

Ankit Pai N – 2462036

Joshua Zachary Jose – 2462093

Jeevitha A – 2462079

R Karthik – 2462131

Batch 59

GIT HUB LINK : <https://github.com/Ankitpaiii/job-portal>



Table of Contents

1. Abstract
2. Introduction
3. Scope of the Project
4. Tools & Technologies Used
5. System Architecture Overview
6. CIA 2 – Setup, TypeScript Fundamentals & Component Design
 - 6.1 TypeScript Fundamentals
 - 6.2 Angular Architecture & Component Design
7. CIA 3 – Advanced Angular Features
 - 7.1 Routing and Navigation
 - 7.2 Services and Dependency Injection
 - 7.3 Forms and Validation
 - 7.4 Custom Pipes and Directives
 - 7.5 Angular Material and UI Design
 - 7.6 Observables, HTTP Client, and Error Handling
 - 7.7 Integration and Testing
8. Module-Wise Description with Screenshots
9. Code Structure Reference
10. Challenges Faced & Solutions
11. Outcome
12. Future Enhancements
13. Conclusion
14. References



1. Abstract

The Job Portal and Recruitment Management System is a modern Single Page Application (SPA) developed using Angular and TypeScript to streamline the recruitment process between job seekers and recruiters. The application provides an interactive platform where candidates can explore job opportunities, apply for positions, and recruiters can manage job postings and applicant data efficiently.

The project leverages Angular's component-based architecture, routing, services, dependency injection, reactive forms, custom pipes, custom directives, HTTP interceptors, and route guards to build a scalable and maintainable system. Angular Material ensures a consistent, responsive, and professional user interface. A mock backend using JSON Server simulates real-time data persistence and CRUD operations, enabling a realistic enterprise-level workflow without requiring a full backend setup.

2. Introduction

In today's digital-first hiring environment, recruitment platforms play a crucial role in connecting organizations with suitable talent. Traditional recruitment systems often suffer from poor usability, lack of real-time feedback, and inefficient data management.

This project addresses these issues by developing a Job Portal and Recruitment Management System that provides a clean user experience, structured job listings, dynamic application handling, and recruiter dashboards using modern Angular development practices.

Problem Statement

Conventional job portals are often cluttered, slow, and lack responsive interaction. Job seekers face difficulty in filtering relevant jobs, while recruiters struggle with managing applications efficiently. There is a need for a lightweight, reactive, and user-friendly recruitment platform.

Proposed Solution

The proposed solution is a web-based SPA that offers:

- An intuitive job search and application interface for candidates
- A centralized dashboard for recruiters to manage job postings and applications
- Real-time UI updates using Angular services and observables
- A scalable frontend architecture suitable for future backend integration



- Route guards and interceptors for secure, error-resilient operation

Objectives

- To design and develop a scalable Job Portal using Angular and TypeScript
- To implement modular component-based architecture
- To utilize Angular routing with child routes and guards for seamless navigation
- To perform CRUD operations using a mock backend (JSON Server)
- To apply reactive and template-driven forms with proper validation
- To build a responsive UI using Angular Material components including Dialogs
- To implement custom pipes and directives for enhanced functionality
- To handle errors globally using HTTP interceptors

3. Scope of the Project

- Job listing and job detail viewing with dynamic routing
- Job application submission by candidates with confirmation dialogs
- Recruiter login with role-based authentication and route protection
- Recruiter dashboard with child routes for posted jobs and received applications
- Search and filter functionality using custom pipes and component logic
- Mock backend data persistence using JSON Server
- View count tracking per job listing
- Global HTTP error interception and user notification

4. Tools & Technologies Used

Tool / Technology	Purpose
Angular v21	Frontend SPA framework
TypeScript	Type-safe application logic
Angular Material	Professional UI components and theming
Angular CDK	Component Dev Kit for Material behaviors
RxJS	Reactive programming and Observables

Tool / Technology	Purpose
Node.js (v18+)	Runtime environment
Angular CLI	Project scaffolding, build, and dev server
JSON Server	Mock REST API backend
HTML5 / SCSS	Structure and styling
Visual Studio Code	Development environment

5. System Architecture Overview

Directory Structure

```

src/
├── app/
│   ├── components/
│   │   ├── navbar/           # Global navigation with auth state
│   │   ├── job-list/         # Job browsing with filters & highlight directive
│   │   ├── job-detail/       # Individual job view with route params
│   │   ├── apply-job/        # Reactive form + MatDialog confirmation
│   │   ├── login/           # Template-driven form + mock auth
│   │   ├── post-job/         # Reactive form for recruiter job posting
│   │   ├── company-dashboard/ # Parent component with RouterOutlet
│   │   │   ├── posted-jobs.ts # Child route component
│   │   │   └── recent-applications.ts # Child route component
│   │   └── confirm-dialog/    # Reusable Material Dialog component
│   ├── services/
│   │   ├── job.service.ts
│   │   ├── applicant.service.ts
│   │   └── company.service.ts
│   ├── models/
│   │   ├── job.model.ts
│   │   ├── applicant.model.ts
│   │   ├── application.model.ts
│   │   └── company.model.ts
│   ├── guards/
│   │   └── auth.guard.ts
│   ├── interceptors/
│   │   └── error.interceptor.ts
│   ├── directives/
│   │   └── highlight.directive.ts
│   ├── pipes/
│   │   └── filter.pipe.ts
│   ├── app.routes.ts
│   └── app.config.ts
├── styles.scss
└── index.html

```

6. CIA 2 – Setup, TypeScript Fundamentals & Component Design

6.1 TypeScript Fundamentals



The project is built entirely in TypeScript, using its advanced type system to define robust data models.

6.1.1 Interfaces and Models

Four core TypeScript interfaces are defined in the `models/` directory:

`job.model.ts`

```
export interface Job {
  id: number;
  title: string;
  location: string;
  type: string;      // 'Full Time', 'Part Time', etc.
  salary: number;
  experience: string;
  description: string;
  companyId: number;
  postedDate: string;
  views?: number;
  applications?: Application[];
}
```

`applicant.model.ts`

```
export interface Applicant {
  id?: number;
  name: string;
  email: string;
  phone: string;
}
```

`application.model.ts`

```
export interface Application {
  id?: number;
  jobId: number;
  applicantId: number;
  name: string;
  email: string;
  phone: string;
  experience: string;
  coverLetter?: string;
  applicationDate: string;
  status: 'pending' | 'reviewed' | 'accepted' | 'rejected';
}
```

`company.model.ts`

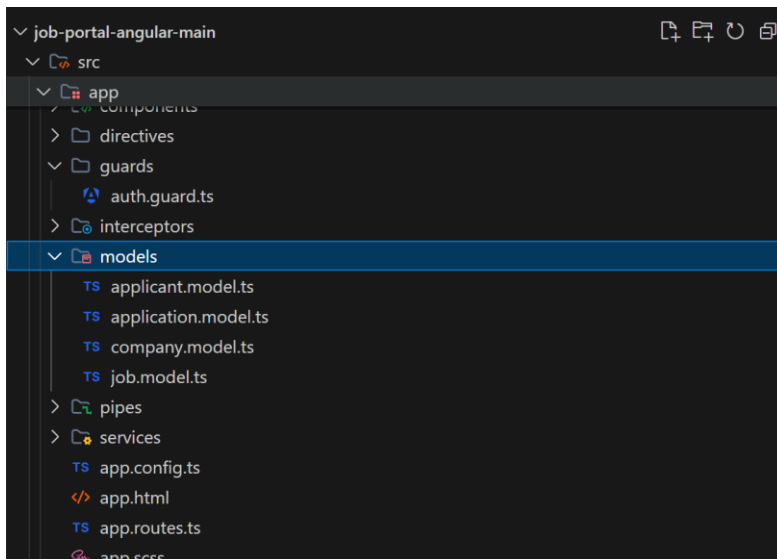
```
export interface Company {
  id?: number;
  name: string;
```



```
email: string;  
industry?: string;  
}
```

TypeScript Features Used

Feature	Where Used
Interfaces	All 4 model files
Readonly properties	Service API URLs
Generics	Observable<Job[]>, Observable<Application>
Optional properties	Job.views?, Application.id?
Union types	'pending' 'reviewed' 'accepted' 'rejected'
Type inference	Component property declarations



6.2 Angular Architecture & Component Design

6.2.1 Component List

Component	Selector	Role
App	app-root	Root shell with <router-outlet>
NavbarComponent	app-navbar	Top navigation bar with auth state
JobList	app-job-list	Browsable job listings grid
JobDetail	app-job-detail	Full job detail view
ApplyJob	app-apply-job	Application submission form
Login	app-login	Recruiter/candidate login
PostJob	app-post-job	Job posting form for recruiter
CompanyDashboard	app-company-dashboard	Recruiter dashboard shell
PostedJobs	app-posted-jobs	Child: list of company's jobs
RecentApplications	app-recent-applications	Child: applicants for a job
ConfirmDialog	app-confirm-dialog	Reusable confirmation modal

6.2.2 Data Binding

One-way (Interpolation):

```
<mat-card-title>{{ job.title }}</mat-card-title>
<span>{{ job.salary | currency:'INR':'symbol':'1.0-0' }}</span>
```

Two-way (ngModel):

```
<input matInput [(ngModel)]="searchTitle" name="email" required>
```

Property Binding:

```
<button [disabled]="!email">LOGIN</button>
<mat-card [appHighlight]="job.id % 2 === 0">...</mat-card>
```

Event Binding:

```
<button (click)="clearFilters()">Clear</button>
<form (ngSubmit)="onSubmit()">...</form>
```

6.2.3 Structural Directives

```
<!-- *ngFor: Iterating job list -->
<mat-card *ngFor="let job of filteredJobs">...</mat-card>

<!-- *ngIf: Conditional rendering -->
<div *ngIf="isLoading"><mat-spinner></mat-spinner></div>
<div *ngIf="errorMessage">{{ errorMessage }}</div>
```

Filters

 Search Title

Location

All Locations

Job Type

All Types

10 Available Jobs

Explore all open positions

Angular Developer

₹80,000

Tech Company #1

Bangalore

Full Time

Jan 10, 2026

We are looking for an experienced Angular Developer to join our team.....

VIEW DETAILS

APPLY NOW

Frontend Engineer

FEATURED
₹20,000

Tech Company #2

Chennai

Internship

Dec 31, 2025

Great opportunity for freshers to learn frontend technologies.....

VIEW DETAILS

APPLY NOW

Backend Developer

₹120,000

Tech Company #3

Bangalore

Full Time

Dec 31, 2025

We are looking for an experienced Backend Developer to join our team.....

VIEW DETAILS

APPLY NOW

7. CIA 3 – Advanced Angular Features

7.1 Routing and Navigation

7.1.1 Route Configuration

```
export const routes: Routes = [
  { path: '', redirectTo: 'jobs', pathMatch: 'full' },
  { path: 'jobs', component: JobList },
  { path: 'jobs/:id', component: JobDetail },
  { path: 'apply/:jobId', component: ApplyJob },
  {
    path: 'dashboard',
    component: CompanyDashboard,
    canActivate: [AuthGuard],
    children: [
      { path: '', redirectTo: 'posted-jobs', pathMatch: 'full' },
      { path: 'posted-jobs', component: PostedJobs },
      { path: 'applications/:jobId', component: RecentApplications }
    ]
  },
  { path: 'post-job', component: PostJob, canActivate: [AuthGuard] },
  { path: 'login', component: Login },
];
```

7.1.2 Route Parameters

In JobDetail, the ActivatedRoute service reads the :id parameter to dynamically fetch the correct job:

```
this.route.paramMap.subscribe(params => {  
  const id = Number(params.get('id'));  
  this.loadJob(id);  
});
```

7.1.3 Child Routes

The CompanyDashboard component acts as a parent layout using <router-outlet>:

```
<!-- company-dashboard.html -->  
<div class="content-section">  
  <router-outlet></router-outlet> <!-- Child components render here -->  
</div>
```

URL	Component Rendered
/dashboard	Redirects to /dashboard/posted-jobs
/dashboard/posted-jobs	PostedJobs (child)
/dashboard/applications/3	RecentApplications (child, for job ID 3)

7.1.4 Route Guard (AuthGuard)

```
@Injectable({ providedIn: 'root' })  
export class AuthGuard implements CanActivate {  
  canActivate(): boolean | UrlTree {  
    const userString = localStorage.getItem('user');  
    if (userString) {  
      const user = JSON.parse(userString);  
      if (user.role === 'company') return true;  
    }  
    this.router.navigate(['/login']);  
    return false;  
  }  
}
```

```
src > app > TS app.routes.ts > ...
1 import { Routes } from '@angular/router';
2 import { JobList } from './components/job-list/job-list';
3 import { JobDetail } from './components/job-detail/job-detail';
4 import { ApplyJob } from './components/apply-job/apply-job';
5 import { CompanyDashboard } from './components/company-dashboard/company-dashboard';
6 import { Login } from './components/login/login';
7 import { PostJob } from './components/post-job/post-job';
8 import { AuthGuard } from './guards/auth.guard';
9 import { PostedJobs } from './components/company-dashboard/posted-jobs';
10 import { RecentApplications } from './components/company-dashboard/recent-applications';
11
12 export const routes: Routes = [
13   { path: '', redirectTo: 'jobs', pathMatch: 'full' },
14   { path: 'jobs', component: JobList },
15   { path: 'jobs/:id', component: JobDetail },
16   { path: 'apply/:jobId', component: ApplyJob },
17   {
18     path: 'dashboard',
19     component: CompanyDashboard,
20     canActivate: [AuthGuard],
21     children: [
22       { path: '', redirectTo: 'posted-jobs', pathMatch: 'full' },
23       { path: 'posted-jobs', component: PostedJobs },
24       { path: 'applications/:jobId', component: RecentApplications }
25     ]
26   },
27   { path: 'post-job', component: PostJob, canActivate: [AuthGuard] },
28   { path: 'login', component: Login },
29 ];
30
```

7.2 Services and Dependency Injection

7.2.1 JobService

```
@Injectable({ providedIn: 'root' })
export class JobService {
  private apiUrl = 'http://localhost:3000/jobs';
  constructor(private http: HttpClient) {}

  getJobs(): Observable<Job[]> { return this.http.get<Job[]>(this.apiUrl); }
  getJobById(id: number): Observable<Job> { return this.http.get<Job>(`${this.apiUrl}/${id}`); }
  addJob(job: Job): Observable<Job> { return this.http.post<Job>(this.apiUrl, { ...job, views: 0 }); }
  getJobsByCompany(id: number) { return this.http.get<Job[]>(this.apiUrl); }
  incrementJobView(job: Job) { return this.http.patch<Job>(this.apiUrl, { ...job, views: job.views + 1 }); }
}
```

7.2.2 ApplicantService

Handles candidate registration, login lookup, and application management.

```
applyJob(application: Application): Observable<Application> {
  return this.http.post<Application>(this.applicationsUrl, application);
}

getApplicationsByJob(jobId: number): Observable<Application[]> {
```



```
    return this.http.get<Application[]>(`${this.applicationsUrl}?jobId=${jobId}`);  
  }  
}
```

7.2.3 CompanyService

Manages company records including login (email lookup) and registration.

7.2.4 Dependency Injection

Services are injected into components via constructor injection. Angular's `providedIn: 'root'` ensures a single shared instance (singleton pattern) across the application:

```
constructor(  
  private jobService: JobService,  
  private router: Router,  
  private snackBar: MatSnackBar  
) {}
```

```
src > app > services > job.service.ts > ...  
1  import { Injectable } from '@angular/core';  
2  import { HttpClient } from '@angular/common/http';  
3  import { Observable } from 'rxjs';  
4  import { Job } from '../models/job.model';  
5  
6  @Injectable({  
7    providedIn: 'root'  
8  })  
9  export class JobService {  
10     private apiUrl = 'http://localhost:3000/jobs';  
11  
12     constructor(private http: HttpClient) { }  
13  
14     getJobs(): Observable<Job[]> {  
15       return this.http.get<Job[]>(this.apiUrl);  
16     }  
17  
18     getJobById(id: number): Observable<Job> {  
19       return this.http.get<Job>(`${this.apiUrl}/${id}`);  
20     }  
21  
22     addJob(job: Job): Observable<Job> {  
23       // Initialize views to 0 for new jobs  
24       const jobWithViews = { ...job, views: 0 };  
25       return this.http.post<Job>(this.apiUrl, jobWithViews);  
26     }  
27  
28     getJobsByCompany(companyId: number): Observable<Job[]> {  
29       // Embed applications to get real counts  
30       return this.http.get<Job[]>(`${this.apiUrl}?companyId=${companyId}&embed=applications`);  
31     }  
32  
33     incrementJobView(job: Job): Observable<Job> {  
34       const updatedViews = (job.views || 0) + 1;  
35       return this.http.patch<Job>(`${this.apiUrl}/${job.id}`, { views: updatedViews });  
36     }  
37   }  
38 }
```

7.3 Forms and Validation

7.3.1 Template-Driven Form – Login Component

```
<form (ngSubmit)="onLogin()">  
  <mat-form-field appearance="fill">  
    <mat-label>Email</mat-label>  
    <input matInput [(ngModel)]="email" name="email" required>  
  </mat-form-field>
```



```
<mat-form-field appearance="fill">
  <mat-label>Role</mat-label>
  <mat-select [(ngModel)]="role" name="role">
    <mat-option value="applicant">Job Seeker</mat-option>
    <mat-option value="company">Recruiter</mat-option>
  </mat-select>
</mat-form-field>
<button mat-raised-button color="primary" [disabled]="!email">LOGIN</button>
</form>
```

7.3.2 Reactive Forms – Apply Job Component

```
this.applyForm = this.fb.group({
  name:    ['', Validators.required],
  email:   ['', [Validators.required, Validators.email]],
  phone:   ['', Validators.required],
  experience: ['', Validators.required],
  coverLetter: ['']
});
```

7.3.3 Reactive Forms – Post Job Component

```
this.jobForm = this.fb.group({
  title:    ['', Validators.required],
  location: ['', Validators.required],
  type:     ['', Validators.required],
  salary:   ['', Validators.required],
  experience: ['', Validators.required],
  description: ['', Validators.required],
});
```

7.3.4 Validation Feedback

- [disabled]="!email" on submit button prevents invalid submission
- Validators.email ensures proper email format
- MatSnackBar shows success/error messages after submission

Login

Email*

admin@gmail.com

Role

Recruiter

LOGIN



Apply for Job

Submit your application



Full Name*



Email*



Phone Number*



Years of Experience*

Resume / CV



Drag & drop or [browse](#) to upload

Cover Letter

CANCEL

SUBMIT APPLICATION

Post a New Job

POST JOB CANCEL

7.4 Custom Pipes and Directives

7.4.1 Custom Filter Pipe — filter.pipe.ts

```
@Pipe({ name: 'filter', standalone: true })
export class FilterPipe implements PipeTransform {
  transform(items: any[], searchText: string, field: string): any[] {
    if (!items) return [];
    if (!searchText) return items;
    return items.filter(it =>
      it[field].toLowerCase().includes(searchText.toLowerCase())
    );
  }
}
```

Usage in template:

```
{{ jobs | filter:searchTitle:'title' }}
```

7.4.2 Custom Highlight Directive — highlight.directive.ts

```
@Directive({ selector: '[appHighlight]', standalone: true })
export class HighlightDirective implements OnInit {
```

```
@Input('appHighlight') isFeatured: boolean = false;
```

```
ngOnInit() {
  if (this.isFeatured) {
    this.renderer.setStyle(this.el.nativeElement, 'border', '2px solid #3f51b5');
    this.renderer.setStyle(this.el.nativeElement, 'background-color', '#f0f2ff');
    // Adds 'FEATURED' badge text in top-right corner
  }
}
```

7.4.3 Built-in Pipes Used

Pipe	Usage Example
CurrencyPipe	{{ job.salary currency:'INR':'symbol':'1.0-0' }}
DatePipe	{{ job.postedDate date }}
SlicePipe	{{ job.description slice:0:120 }}
UpperCasePipe	{{ app.status uppercase }}

```
src > app > pipes > filter.pipe.ts > ...
1  import { Pipe, PipeTransform } from '@angular/core';
2
3  @Pipe({
4    name: 'filter',
5    standalone: true
6  })
7  export class FilterPipe implements PipeTransform {
8    transform(items: any[], searchText: string, field: string): any[] {
9      if (!items) return [];
10     if (!searchText) return items;
11
12     searchText = searchText.toLowerCase();
13
14     return items.filter(it => {
15       return it[field].toLowerCase().includes(searchText);
16     });
17   }
18 }
```

7.5 Angular Material and UI Design

7.5.1 Material Components Used

Component	Used In
MatCard	Job listings, Job Detail, Dashboard stats
MatTable	Applications list in dashboard
MatToolbar	Navbar
MatFormField / MatInput	All forms
MatButton / MatIconButton	All action buttons
MatIcon	Navigation, chips, action icons
MatDialog	Application confirmation popup
MatSnackBar	Success and error notifications
MatSelect	Dropdowns for job type, location, role
MatAutocomplete	Job title search suggestions
MatProgressSpinner	Loading indicators
MatDivider	Section separators

7.5.2 Material Dialog for Confirmation

```
const dialogRef = this.dialog.open(ConfirmDialog, {
  width: '350px',
  data: {
    title: 'Confirm Application',
    message: 'Are you sure you want to submit your application?'
  }
});
```

```
dialogRef.afterClosed().subscribe(result => {
  if (result) {
    // Submit application via service
  }
});
```

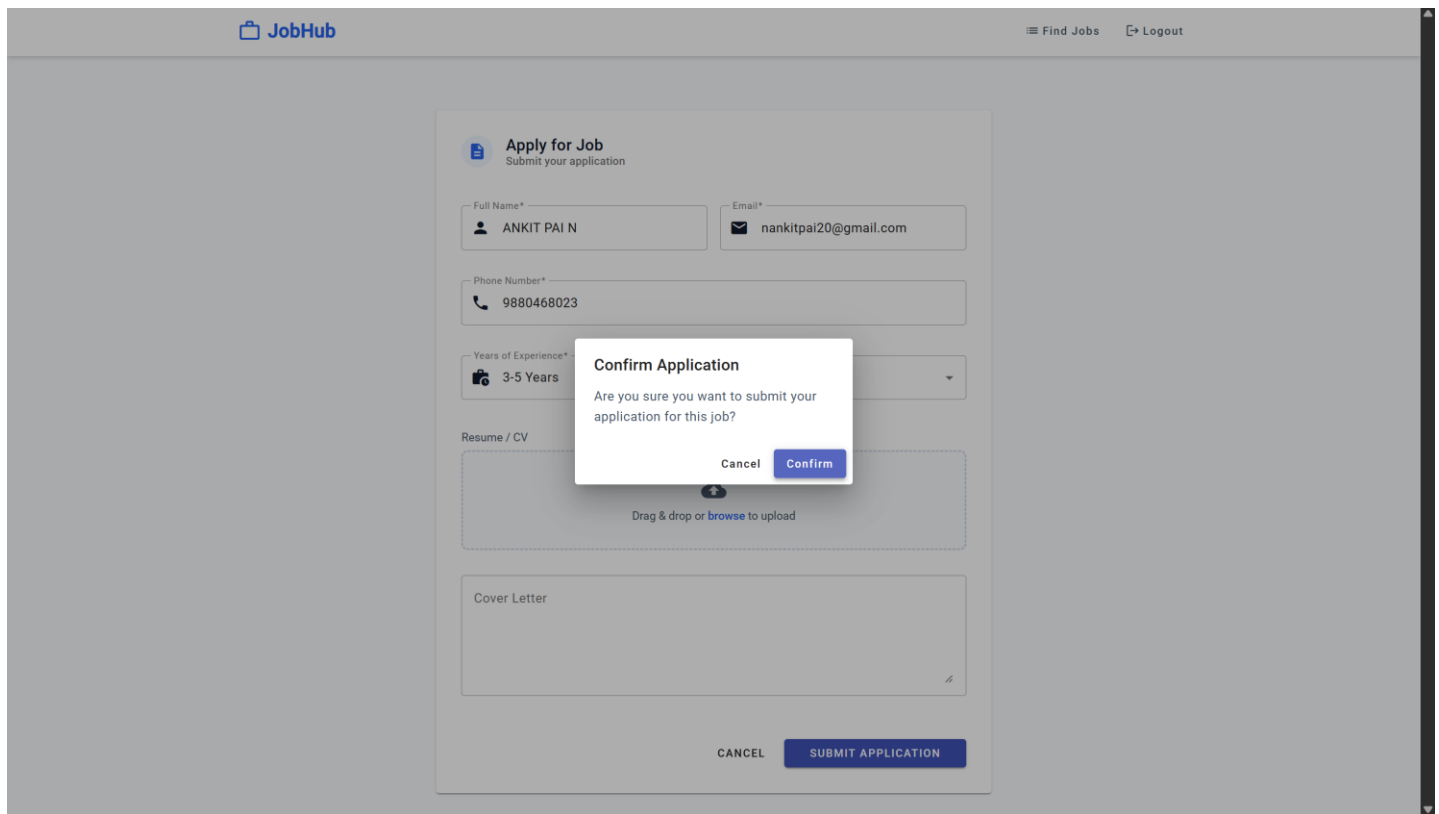
ConfirmDialog template:

```
<h2 mat-dialog-title>{{ data.title }}</h2>
<mat-dialog-content><p>{{ data.message }}</p></mat-dialog-content>
<mat-dialog-actions align="end">
  <button mat-button (click)="onNoClick()">Cancel</button>
  <button mat-raised-button color="primary" [mat-dialog-close]="true">Confirm</button>
</mat-dialog-actions>
```

7.5.3 Theming

The prebuilt indigo-pink Material theme is applied globally, with a custom CSS variable system:

```
:root {
  --primary-color: #2563eb;
  --accent-color: #f59e0b;
  --background-bg: #f8fafc;
}
```



The screenshot shows a web application interface for 'JobHub'. At the top, there's a navigation bar with 'JobHub' on the left and 'Find Jobs' and 'Logout' on the right. The main content area features a 'Apply for Job' form. The form has several input fields: 'Full Name*' (filled with 'ANKIT PAI N'), 'Email*' (filled with 'nankitpai20@gmail.com'), 'Phone Number*' (filled with '9880468023'), and 'Years of Experience*' (a dropdown menu showing '3-5 Years'). Below these is a 'Resume / CV' section with a dashed border and a 'Cover Letter' text area. A modal dialog titled 'Confirm Application' is centered over the form, asking 'Are you sure you want to submit your application for this job?' with 'Cancel' and 'Confirm' buttons. At the bottom of the form, there are 'CANCEL' and 'SUBMIT APPLICATION' buttons.

7.6 Observables, HTTP Client, and Error Handling

7.6.1 RxJS Observables

All data fetching is handled asynchronously using `Observable<T>` from RxJS:

```
ngOnInit(): void {
  this.isLoading = true;
  this.jobService.getJobs().subscribe({
    next: (data) => {
      this.jobs = data;
    }
  });
}
```

```

        this.isLoading = false;
    },
    error: (e) => {
        this.errorMessage = 'Failed to load jobs.';
        this.isLoading = false;
    }
});
}

```

7.6.2 HTTP Interceptor for Global Error Handling

```

export const ErrorInterceptor: HttpInterceptorFn = (req, next) => {
    const snackBar = inject(MatSnackBar);

    return next(req).pipe(
        catchError((error: HttpResponse) => {
            let errorMessage = 'An unknown error occurred!';
            if (error.status === 0) {
                errorMessage = 'Cannot connect to server. Is JSON Server running?';
            } else {
                errorMessage = `Error ${error.status}: ${error.message}`;
            }
            snackBar.open(errorMessage, 'Close', {
                duration: 5000,
                panelClass: ['error-snackbar']
            });
            return throwError(() => error);
        })
    );
};

```

Registered globally in app.config.ts:

```

provideHttpClient(withInterceptors([ErrorInterceptor]))

```

7.6.3 View Count Tracking with Observables

```

this.jobService.incrementJobView(data).subscribe({
    next: () => console.log('View count incremented'),
    error: (e) => console.error('Failed to increment view count', e)
});

```

```
src > app > interceptors > TS error.interceptor.ts > ...
1 import { HttpInterceptorFn, HttpResponse } from '@angular/common/http';
2 import { inject } from '@angular/core';
3 import { MatSnackBar } from '@angular/material/snack-bar';
4 import { catchError, throwError } from 'rxjs';
5
6 export const ErrorInterceptor: HttpInterceptorFn = (req, next) => {
7   const snackBar = inject(MatSnackBar);
8
9   return next(req).pipe(
10     catchError((error: HttpResponse) => {
11       let errorMessage = 'An unknown error occurred!';
12
13       if (error.error instanceof ErrorEvent) {
14         // Client-side error
15         errorMessage = `Error: ${error.error.message}`;
16       } else {
17         // Server-side error
18         errorMessage = `Error Code: ${error.status}\nMessage: ${error.message}`;
19         if (error.status === 0) {
20           errorMessage = 'Cannot connect to server. Please check if JSON server is running.';
21         }
22       }
23
24       snackBar.open(errorMessage, 'Close', {
25         duration: 5000,
26         horizontalPosition: 'end',
27         verticalPosition: 'bottom',
28         panelClass: ['error-snackbar']
29       });
30
31       return throwError(() => error);
32     })
33   );
34 };

```

7.7 Integration and Testing

7.7.1 Full Application Integration Flow

User visits /jobs

- JobList component calls JobService.getJobs()
- HttpClient sends GET http://localhost:3000/jobs
- ErrorInterceptor wraps the request
- JSON Server returns jobs array
- Component renders job cards with HighlightDirective
- FilterPipe enables real-time search

User clicks APPLY NOW

- Navigate to /apply/:jobId
- ApplyJob component renders Reactive Form
- User fills form → Clicks Submit
- MatDialog ConfirmDialog opens
- User confirms → ApplicantService.applyJob() called
- POST to http://localhost:3000/applications
- SnackBar shows 'Application Submitted!'



→ Navigate back to /jobs

Recruiter logs in → localStorage stores {role: 'company'}

→ Navbar shows 'Dashboard' and 'Logout' buttons

→ Navigate to /dashboard (AuthGuard allows access)

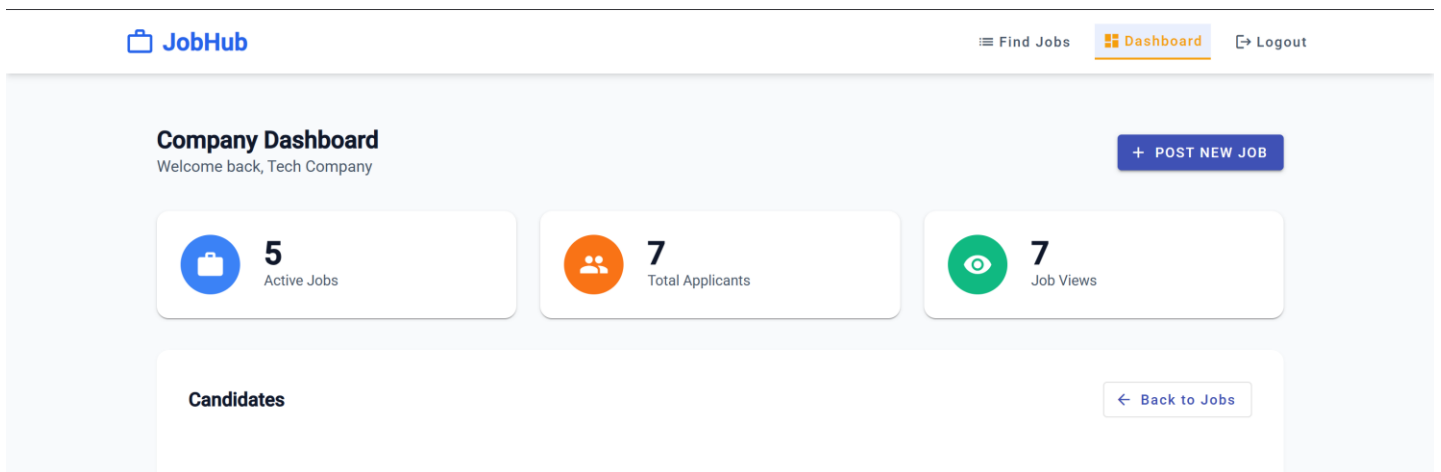
→ Child route /dashboard/posted-jobs loads PostedJobs

→ Click 'View Candidates' → /dashboard/applications/:jobId

→ RecentApplications fetches and displays candidates in MatTable

7.7.2 Navigation Testing

Test Case	Expected Result	Status
Open /jobs	Job cards visible	Pass
Click a job card	Navigate to /jobs/:id	Pass
Click 'Apply Now'	Navigate to /apply/:jobId	Pass
Submit application (valid)	Dialog → Snackbar success	Pass
Visit /dashboard (not logged in)	Redirect to /login	Pass
Login as Recruiter	Redirect to /dashboard	Pass
Stop JSON Server → Load jobs	Error Snackbar appears	Pass
Filter by Location dropdown	Job list updates	Pass



```
( ) db.json > ...
1 {
2   "jobs": [
3     {
4       "id": 1,
5       "title": "Angular Developer",
6       "company": "TCS",
7       "location": "Bangalore",
8       "type": "Full Time",
9       "experience": "2-4 Years",
10      "salary": "80000",
11      "postedDate": "2026-01-10",
12      "description": "We are looking for an experienced Angular Developer to join our team...",
13      "companyId": 1,
14      "views": 5
15    },
16    {
17      "id": 2,
18      "title": "Frontend Engineer",
19      "company": "Infosys",
20      "location": "Chennai",
21      "type": "Internship",
22      "experience": "0-1 Year",
23      "salary": "20000",
24      "postedDate": "2025-12-31",
25      "description": "Great opportunity for freshers to learn frontend technologies...",
26      "companyId": 2,
27      "views": 0
28    },
29    {
30      "id": 3,
31      "title": "Backend Developer",
32      "company": "Accenture",
33      "location": "Hyderabad",
```

8. Module-Wise Description with Screenshots

8.1 Landing Page & Navigation

The NavbarComponent provides application-wide navigation. It reads the user session from localStorage and conditionally renders navigation links.

- Shows 'Dashboard' and 'Logout' only when logged in as a Recruiter
- Shows 'Log In' button for unauthenticated users

8.2 Job Listing Module

Displays all job postings dynamically using *ngFor and Angular Material cards. Features include:

- Job title search with autocomplete suggestions
- Filter by Location (dropdown)
- Filter by Job Type (dropdown)
- View count display
- 'Featured' job highlighting via custom directive

Filters Clear

Location

Bangalore

Job Type

Part Time

1 Matching Jobs

Found matches for your search criteria

UI/UX Designer

FEATURED

₹60,000

Tech Company #6

Bangalore

Part Time

Jan 16, 2026

Creative designer needed for our food delivery app.....

VIEW DETAILS

APPLY NOW

8.3 Search and Filter Functionality

Filtering is performed via component logic combined with the custom FilterPipe. Users can:

- Search by job title (with autocomplete)
- Filter by city/location
- Filter by job type (Full Time, Part Time, Remote, etc.)
- Clear all filters with one click

8.4 Job Detail View

Uses dynamic routing (/jobs/:id) to display job-specific information. Automatically increments the view counter on page load via a PATCH request.



Angular Developer

Tech Company #1 • Bangalore

[Apply Now](#)

Job Description

We are looking for an experienced Angular Developer to join our team...

Requirements

- Experience with Angular and TypeScript.
- Understanding of REST APIs and HTTP.
- Strong problem-solving skills.
- Ability to work in a team.

Benefits

- Competitive Salary
- Remote Friendly
- Health Insurance

Job Overview

SALARY
\$ ₹80,000

POSTED DATE
Jan 10, 2026

JOB TYPE
Full Time

LOCATION
Bangalore

[APPLY FOR THIS JOB](#)
[← Back to Jobs](#)

8.5 Job Application Module

Implements a reactive form with validators. On submission, a MatDialog opens for confirmation before the data is posted to the JSON Server.

8.6 Recruiter Authentication

A template-driven login form allows the user to select a Role (Job Seeker or Recruiter). The session is stored in localStorage. Route Guards prevent unauthorized access to recruiter routes.

8.7 Recruiter Dashboard with Child Routes



The dashboard is divided into two child route views:

- /dashboard/posted-jobs: Grid of all jobs posted by the company
- /dashboard/applications/:jobId: Table of candidates who applied for a specific job

8.8 Data Persistence (JSON Server)

All data is persisted in db.json. The JSON Server provides a full REST API (GET, POST, PATCH, DELETE) on port 3000.

```
{
  "jobs": [ { "id": 1, "title": "Frontend Developer", "views": 12, ... } ],
  "companies": [ { "id": 1, "name": "Tech Corp" } ],
  "applications": [ { "id": 1, "jobId": 1, "status": "pending", ... } ]
}
```

```
PS C:\Users\nanki\Downloads\Compressed\job-portal-angular-main>
PS C:\Users\nanki\Downloads\Compressed\job-portal-angular-main> npm run server
>>

> job-portal@0.0.0 server
> json-server --watch db.json

\{^_^}/ hi!
\{^_^}/ hi!

Loading db.json
Done

Resources
http://localhost:3000/jobs
http://localhost:3000/companies
http://localhost:3000/applications

Home
http://localhost:3000
```

9. Code Structure Reference

```

job-portal-angular-main/
├── src/
│   ├── app/
│   │   ├── app.config.ts    ← Global providers, Router, HttpClient
│   │   ├── app.routes.ts   ← All route definitions
│   │   ├── app.ts          ← Root component
│   │   ├── components/
│   │   │   ├── navbar/
│   │   │   │   ├── navbar.ts    ← Auth state, logout logic
│   │   │   │   └── navbar.html  ← Conditional nav links
│   │   │   ├── job-list/
│   │   │   ├── job-detail/
│   │   │   ├── apply-job/
│   │   │   ├── login/
│   │   │   ├── post-job/
│   │   │   ├── company-dashboard/
│   │   │   │   ├── posted-jobs.ts
│   │   │   │   └── recent-applications.ts
│   │   │   └── confirm-dialog/
│   │   ├── services/
│   │   ├── models/
│   │   ├── guards/
│   │   ├── interceptors/
│   │   ├── directives/
│   │   └── pipes/
│   ├── styles.scss
│   └── index.html
├── db.json
└── package.json
  
```

10. Challenges Faced & Solutions

Challenge	Solution
Component communication across pages	Used Angular Services with providedIn: 'root' singleton pattern
Decentralized error handling	Implemented global ErrorInterceptor with MatSnackBar
Protecting recruiter routes	Implemented AuthGuard with CanActivate interface
Flat dashboard structure	Refactored to child routes with RouterOutlet

Challenge	Solution
Form validation complexity	Used ReactiveFormsModule with FormBuilder and Validators
Application confirmation UX	Used MatDialog to provide professional confirmation modals
View count tracking on page load	Used PATCH request in JobService.incrementJobView()
Featured job highlighting	Created custom HighlightDirective with Renderer2

11. Outcome

- Successfully developed a fully functional Job Portal SPA using Angular v21
- Implemented all 9 advanced Angular concepts from CIA 3 requirements
- Achieved modular and maintainable codebase with standalone components
- Created a responsive and user-friendly interface using Angular Material
- Demonstrated real-world Angular patterns: guards, interceptors, custom pipes, custom directives, child routes, and Material Dialogs
- Integrated mock REST API using JSON Server with CRUD data operations
- Applied TypeScript interfaces, generics, and type safety throughout the project

12. Future Enhancements

- Backend integration using Node.js / Spring Boot REST APIs
- Authentication using JWT tokens with secure HTTP-only cookies
- Resume upload and automated PDF generation
- AI-based job recommendation engine
- Real-time notifications using WebSockets
- Cloud database integration (Firebase / MongoDB Atlas)
- Admin panel for platform management
- Email notification system for application status updates

13. Conclusion

The Job Portal and Recruitment Management System successfully demonstrates the comprehensive use of Angular and TypeScript in building scalable, data-driven web applications. The project covers the full spectrum of modern Angular development.

From foundational concepts (components, directives, pipes, services) to advanced features (route guards, child routes,



HTTP interceptors, reactive forms with Material Dialogs), the application simulates a real-world enterprise recruitment platform.

The use of JSON Server as a mock backend allowed for realistic CRUD operations without a dedicated server, while Angular Material ensured a professional and responsive user experience across devices.

This project solidified the understanding of modern Angular architecture, component interaction patterns, and enterprise UI development practices defined in CIA 2 and CIA 3 objectives.

14. References

1. Angular Official Documentation – <https://angular.dev>
2. Angular Material Documentation – <https://material.angular.io>
3. TypeScript Handbook – <https://www.typescriptlang.org/docs/>
4. RxJS Documentation – <https://rxjs.dev/>
5. JSON Server – <https://github.com/typicode/json-server>
6. MDN Web Docs – <https://developer.mozilla.org>
7. GitHub Repository – <https://github.com/Ankitpaiii/job-portal>